

Null 安全

Null安全は、null参照のリスクを大幅に低減するために設計されたKotlinの機能であり、「10億ドルのミス」としても知られています。

Javaを含む多くのプログラミング言語において、最もよくある落とし穴の一つは、null参照のメンバにアクセスすると、null参照例外が発生してしまうことです。Javaでは、これは `NullPointerException`、略して *NPE* に相当します。

Kotlin は、型システムの一部として null 許容性を明示的にサポートしています。つまり、どの変数やプロパティが `null` になり得るかを明示的に宣言することができます。また、null ではない変数を宣言すると、コンパイラはこれらの変数が `null` 値を保持できないように強制し、NPE を防ぎます。

Kotlinのヌル安全性は、実行時ではなくコンパイル時にヌルに関連する潜在的な問題を検出することで、より安全なコードを実現します。この機能は、`null` 値を明示的に表現することで、コードの堅牢性、可読性、および保守性を向上させ、コードの理解と管理を容易にします。

KotlinにおいてNPEが発生する考えられる原因は、以下の2つだけです：

- `throw NullPointerException()` を明示的に呼び出すこと。
- `null`でないことを確認する演算子 `!!` の使い方。
- 初期化中のデータの不整合。例えば、次のような場合：
 - コンストラクタ内で利用可能な初期化されていない `this` が、別の場所で使用されている（`[this のリーク]`）（<https://youtrack.jetbrains.com/issue/KTIJ-9751>）。
 - オープンメンバを呼び出すスーパークラスのコンストラクタで、派生クラスでのその実装が初期化されていない状態を使用している場合。
- Javaとの相互運用：
 - プラットフォーム型の`null`参照のメンバにアクセスしようとした。
 - ジェネリック型におけるNull許容性の問題。例えば、JavaのコードでKotlinの `MutableList<String>` に`null`を追加する場合、これを適切に処理するには `MutableList<String?>` が必要となります。
 - 外部のJavaコードに起因するその他の問題。

NPEのほか、null 安全性に関連するもう 1 つの例外として、`UninitializedPropertyAccessException` があります。Kotlinでは、初期化されていないプロパティにアクセスしようとするこの例外がスローされ、null不可のプロパティが準備が整うまで使用されないようにしています。これは通常、`lateinit` プロパティで発生します。

{style="tip"}

ヌル許容型とヌル非許容型

Kotlinでは、型システムは`null`を保持できる型（ヌル許容型）と、保持できない型（ヌル非許容型）を区別します。たとえば、型が `String` の通常の変数には `null` を格納することはできません：

```
fun main() {  
    //sampleStart
```

```
// 変数にnullではない文字列を代入する
var a: String = "abc"
// nul値ではない変数にnullを再代入しようとしているとコンパイルエラー
a = null
print(a)
// nulは、nul非許容型であるString型の値にはなり得ない
//sampleEnd
}
```

```
{kotlin-runnable="true" validate="false"}
```

a に対して、メソッドを呼び出したりプロパティにアクセスしたりしても問題ありません。**a** はnul非許容の変数であるため、NPE が発生することは絶対にありません。コンパイラは、**a** が常に有効な **String** 値を保持するように保証しているため、**null** の状態でもそのプロパティやメソッドにアクセスしてしまうリスクはありません：

```
fun main() {
//sampleStart
    // 変数にnullではない文字列を代入する
    val a: String = "abc"
    // nul値をとらない変数の長さを返す
    val l = a.length
    print(l)
    // 3
//sampleEnd
}
```

```
{kotlin-runnable="true" validate="false"}
```

null 値を許可するには、変数型の直後に **?** 記号を付けて変数を宣言します。たとえば、**String?** と記述することで、Null許容の文字列を宣言できます。この式により、**String** は **null** を受け入れることができる型になります：

```
fun main() {
//sampleStart
    // 変数にNULL許容の文字列型を割り当てる
    var b: String? = "abc"
    // Null許容の変数にnullを再代入することに成功しました
    b = null
    print(b)
    // null
//sampleEnd
}
```

```
{kotlin-runnable="true"}
```

b に対して **length** を直接呼び出そうとすると、コンパイラはエラーを報告します。これは、**b** がnul許容変数として宣言されており、**null** 値を格納できるためです。nul許容プロパティに直接アクセスしようとする

と、NPE が発生します：

```
fun main() {
//sampleStart
    // 変数にnull許容の文字列型を割り当てる
    var b: String? = "abc"
    // nul許容の変数にnullを再代入する
    b = null
    // nul許容変数の長さを直接返そうとする
    val l = b.length
    // val l = b.!!length //とすると、Exception in thread "main"
    java.lang.NullPointerExceptionが発生
    print(l)
    // String? 型のnul許容レシーバーに対しては、safe (?.) または非nulを保証する (!!.)
    呼び出しのみが許可されます。
//sampleEnd
}
```

```
{kotlin-runnable="true" kotlin-min-compiler-version="2.0" validate="false"}
```

上記の例では、プロパティにアクセスしたり操作を実行したりする前に、nul値の可能性を確認するために、コンパイラはセーフコールを使用することを要求します。 null 許容型を扱う方法はいくつかあります：

- if条件文を使ってnullかどうかを確認する
- セーフコールオペレーター ?.
- エルビス演算子 ?:
- 非Null判定演算子 !!
- Null許容レシーバ
- let 関数
- セーフキャスト as?
- Null許容型コレクション

null 処理のためのツールや手法の詳細や例については、次のセクションをご覧ください。

Check for null with the if conditional

if文を使ってnullかどうかを確認する

null 許容型を扱う際は、NPE を回避するために、null 許容性を安全に処理する必要があります。これに対処する方法の一つは、if条件式を使って明示的にNULL許容性をチェックすることです。

たとえば、b が null かどうかを確認してから、b.length にアクセスしてみてください：

```
fun main() {
//sampleStart
    // null許容な変数にnullを代入する
    val b: String? = null
    // まずNULL許容性を確認してから、lengthにアクセスする
    val l = if (b != null) b.length else -1
}
```

```
    print(1)
    // -1
    //sampleEnd
}
```

```
{kotlin-runnable="true"}
```

上記の例では、コンパイラが **スマートキャスト** を行い、型をヌル許容型の `String?` からヌル非許容型の `String` に変更します。また、実行したチェックに関する情報を追跡し、`if` 条件文内で `length` を呼び出すことも可能にします。

さらに複雑な条件にも対応しています：

```
fun main() {
    //sampleStart
    // 変数にnull許容の文字列を代入する
    val b: String? = "Kotlin"

    // まずNULL許容性を確認してから、lengthにアクセスする
    if (b != null && b.length > 0) {
        print("String of length ${b.length}")
        // String of length 6
    } else {
        // 条件が満たされない場合の代替処理を提供する
        print("Empty string")
    }
    //sampleEnd
}
```

```
{kotlin-runnable="true"}
```

なお、上記の例は、**スマートキャストの前提条件**と同様に、コンパイラがチェックと使用の間に `b` が変化しないことを保証できる場合にのみ機能することに注意してください。

Safe call operator

セーフコール演算子

セーフコール演算子 `?.` を使用すると、ヌル値の扱いをより簡潔な形式で安全に行うことができます。NPEを発生させる代わりに、オブジェクトが `null` の場合、`?.` 演算子は単に `null` を返します：

```
fun main() {
    //sampleStart
    // 変数にnull許容の文字列を割り当てる
    val a: String? = "Kotlin"
    // ヌル可能な変数にnullを割り当てる
    val b: String? = null

    // ヌル許容性を確認し、長さを返すか、ヌルを返す
}
```

```
println(a?.length)
// 6
println(b?.length)
// null
//sampleEnd
}
```

```
{kotlin-runnable="true"}
```

式 `b?.length` は、`b` がヌルでない場合は `b.length` を返し、そうでない場合は `null` を返す、ヌル許容性をチェックする式です。この式の型は `Int?` です。

Kotlin では、`?.` 演算子を `var` 変数と `val` 変数の両方で使用できます：

- ヌル許容の `var` には、`null`（例：`var nullableValue: String? = null`）またはヌル以外の値（例：`var nullableValue: String? = 「Kotlin」`）を格納できます。値が `null` でない場合は、いつでもそれを `null` に変更できます。
- ヌル許容の `val` には、`null`（例：`val nullableValue: String? = null`）またはヌル以外の値（例：`val nullableValue: String? = 「Kotlin」`）を格納できます。値が `null` でない場合、その後でそれを `null` に変更することはできません。訳者注）値が `null` の場合も、その後でそれを `null` 以外に変更することはできません。訳者注）値が初期化されていない場合は、一度だけ `null` か `null` 以外の値を割り当てることができます。

チェーン内では、セーフコールが役立ちます。例えば、ボブは、ある部署に配属される可能性もあるし、そうでない可能性もある従業員です。その部署には、さらに別の従業員が部署の責任者として在籍している場合もあります。ボブの部署の責任者（いる場合）の名前を知るには、次のように入力します：

```
bob?.department?.head?.name
```

このチェーンは、そのプロパティのいずれかが `null` の場合、`null` を返します。

また、代入式の左側に「セーフコール」を配置することもできます：

```
person?.department?.head = managersPool.getManager()
```

上記の例では、セーフコールチェーン内の受信者のいずれかが `null` の場合、代入はスキップされ、右辺の式はまったく評価されません。たとえば、`person` または `person.department` のいずれかが `null` の場合、この関数は呼び出されません。以下は、同じセーフコールに `if` 条件分岐を加えたものになります：

```
if (person != null && person.department != null) {
    person.department.head = managersPool.getManager()
}
```

Elvis operator

エルビス演算子

null 許容型を扱う場合、`null` かどうかを確認し、代替値を指定することができます。たとえば、`b` が `null` でない場合は、`b.length` にアクセスします。それ以外の場合は、別の値を返します：

```
fun main() {
    //sampleStart
    // ヌル可能な変数にnullを割り当てる
    val b: String? = null
    // ヌルかどうかを確認します。ヌルでない場合は長さを返し、ヌルの場合は0を返します。
    val l: Int = if (b != null) b.length else 0
    println(l)
    // 0
    //sampleEnd
}
```

{kotlin-runnable="true"}

`if` 式をすべて記述する代わりに、Elvis 演算子 `?:` を使えば、より簡潔に処理することができます：

```
fun main() {
    //sampleStart
    // Assigns null to a nullable variable
    val b: String? = null
    // ヌルかどうかを確認します。ヌルでない場合は長さを返し、ヌルの場合はヌルでない値を返します
    val l = b?.length ?: 0
    println(l)
    // 0
    //sampleEnd
}
```

{kotlin-runnable="true"}

`?:` の左側の式が `null` でない場合、エルビス演算子はそれ(`?:` の左側の式)を返します。それ以外の場合は、Elvis演算子は右側の式を返します。Elvis演算子の右辺の式は、左辺が `null` の場合にのみ評価されます。

Kotlin では `throw` と `return` は式であるため、エルビス演算子の右辺でもこれらを使用することができます。これは、たとえば関数の引数を確認する際などに便利です：

```
fun foo(node: Node): String? {
    // getParent() をチェックします。null でなければ、それを parent に代入します。null
    // の場合は null を返します。
    val parent = node.getParent() ?: return null
    // getName() をチェックします。null でなければ、name に代入します。null の場合は例
    // 外をスローします
    val name = node.getName() ?: throw IllegalArgumentException("name expected")
}
```

```
// ...  
}
```

Not-null assertion operator

ヌルでないことを保証する演算子

nullでないことを確認する演算子 **!!** は、任意の値をnull不可の型に変換します。

値が **null** ではない変数に **!!** 演算子を適用すると、その変数は null 許容型として安全に扱われ、コードは通常通り実行されます。ただし、値が **null** の場合、**!!** 演算子によってその値が null ではないものとして扱われるため、NPE が発生します。

b が **null** ではなく、**!!** 演算子によって **b** の **null** ではない値（この例では **String**）が返される場合、**length** に正しくアクセスされます：

```
fun main() {  
    //sampleStart  
    // 変数にnull許容の文字列を代入する  
    val b: String? = "Kotlin"  
    // b を null ではないものとみなして、その長さを取得する  
    val l = b!!.length  
    println(l)  
    // 6  
    //sampleEnd  
}
```

{kotlin-runnable="true"}

b が **null** であり、**!!** 演算子によって **b** が **null** ではない値を返すようになった際に、NPE が発生する場合：

```
fun main() {  
    //sampleStart  
    // ヌル可能な変数にnullを代入する  
    val b: String? = null  
    // !!演算子により、b を null ではないものとみなして、その長さにアクセスしようとしま  
    す  
    val l = b!!.length  
    println(l)  
    // Exception in thread "main" java.lang.NullPointerException  
    // NPE例外が生じます。  
    //sampleEnd  
}
```

{kotlin-runnable="true" kotlin-min-compiler-version="2.0" validate="false"}

!! 演算子は、値が `null` ではないと確信しており、NPE が発生する可能性が全くない場合でも、特定の規則によりコンパイラがこれを保証できない場合に特に役立ちます。そのような場合、!! 演算子を使用することで、その値が `null` ではないことをコンパイラに明示的に伝えることができます。

Nullable receiver

Null許容のレシーバー

[null 許容レシーバー型](#)を持つ拡張関数を使用することで、`null` になる可能性のある変数に対してこれらの関数を呼び出すことができます。

`null` 許容型のレシーバー型に対して拡張関数を定義することで、関数を呼び出すたびに `null` チェックを行う代わりに、関数内部で `null` 値を処理することができます。

たとえば、`.toString()` という拡張関数は、`null` 許容のレシーバーに対して呼び出すことができます。`null` 値に対して呼び出された場合、例外をスローすることなく、文字列「`null`」を安全に返します：

```
//sampleStart
fun main() {
    // `person` 変数に格納されているヌル許容型の `Person` オブジェクトに null を代入し
    // ます
    val person: Person? = null

    // ヌル許容の変数 `person` に `.toString` を適用し、文字列を出力する
    println(person.toString())
    // null
}

// 単純な Person クラスを定義します
data class Person(val name: String)
//sampleEnd
```

```
{kotlin-runnable="true"}
```

上記の例では、`person` が `null` であっても、`.toString()` 関数は安全に文字列「`null`」を返します。これは、デバッグやログ出力に役立ちます。

`.toString()` 関数がヌル許容の文字列（文字列表現または `null`）を返す可能性がある場合は、[セーフコール演算子 `?.`](#) を使用してください。`?.` 演算子は、オブジェクトが `null` でない場合にのみ `.toString()` を呼び出し、それ以外の場合は `null` を返します：

```
//sampleStart
fun main() {
    // Assigns a nullable Person object to a variable
    val person1: Person? = null
    val person2: Person? = Person("Alice")

    // Prints "null" if person is null; otherwise prints the result of
    person.toString()
}
```

```
println(person1?.toString())
// null
println(person2?.toString())
// Person(name=Alice)
}

// Person クラスを定義します
data class Person(val name: String)
//sampleEnd
```

```
{kotlin-runnable="true"}
```

The `?.` operator allows you to safely handle potential `null` values while still accessing properties or functions of objects that might be `null`.

Let function

Let関数

`null` 値を処理し、`null` ではない型のみに演算を適用するには、セーフコール演算子 `?.` を `let` 関数 と組み合わせて使用できます。

この組み合わせは、式を評価し、その結果が `null` かどうかを確認し、`null` でない場合にのみコードを実行するのに役立ち、手動での `null` チェックを省くことができます：

```
fun main() {
//sampleStart
    // ヌル許容文字列のリストを宣言します
    val listWithNulls: List<String?> = listOf("Kotlin", null)

    // リスト内の各項目を順に処理する
    for (item in listWithNulls) {
        // 項目がnullかどうかを確認し、nullでない値のみを出力する
        item?.let { println(it) }
    }
    //Kotlin
//sampleEnd
}
```

```
{kotlin-runnable="true"}
```

Safe casts

Kotlinにおける型変換の標準的な演算子は、`as` 演算子です。ただし、オブジェクトがターゲット型でない場合、通常のキャストを行うと例外が発生する可能性があります。

安全な型変換には `as?` 演算子を使用できます。この演算子は、値を指定された型に変換しようと試み、その値が指定された型でない場合は `null` を返します：

```
fun main() {
//sampleStart
    // Any型の変数を宣言します。この変数には、あらゆる型の値を格納できます。
    val a: Any = "Hello, Kotlin!"

    // 『as?』 演算子を使用した、Int への安全なキャスト
    val aInt: Int? = a as? Int
    // 『as?』 演算子を使用した、String への安全なキャスト
    val aString: String? = a as? String

    println(aInt)
    // null
    println(aString)
    // "Hello, Kotlin!"
//sampleEnd
}
```

```
{kotlin-runnable="true"}
```

上記のコードでは、`a` が `Int` ではないため、キャストが安全に失敗し、`null` が出力されます。また、`String?` 型と一致するため、安全なキャストが成功し、「Hello, Kotlin!」が出力されます。

Collections of a nullable type

ヌル許容型のコレクション

`null` になる可能性のある要素のコレクションがあり、`null` ではない要素のみを残したい場合は、`filterNotNull()` 関数を使用します：

```
fun main() {
//sampleStart
    // null値とnull以外の整数値を含むリストを宣言します
    val nullableList: List<Int?> = listOf(1, 2, null, 4)

    // ヌル値を排除し、ヌルではない整数のリストを生成します
    val intList: List<Int> = nullableList.filterNotNull()

    println(intList)
    // [1, 2, 4]
//sampleEnd
}
```

```
{kotlin-runnable="true"}
```

What's next?

次は？

- [JavaとKotlinにおけるヌル許容性の扱い方](#)について学びましょう。
- [確実にヌル非許容](#)なジェネリック型について学びましょう。